{EPITECH}

AREA_

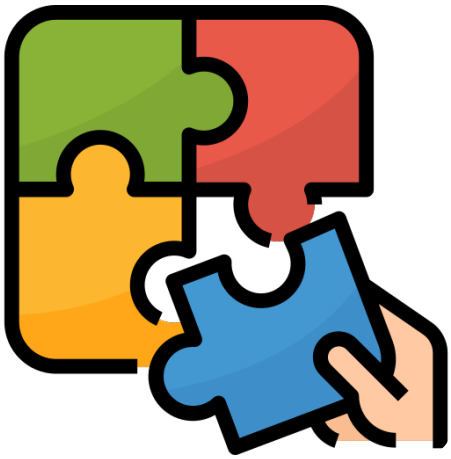
< AREA OF A CIRCLE />



AREA

Quick brief

This is an umbrella project: everything you experimented before comes together inside a single product that needs to work as a coherent whole.



Your role

As a Software Architect, your main goal is neither to reinvent the wheel nor to write numerous lines of code, but rather to understand a need, explore alternatives, make technical decisions, integrate existing components, verify those decisions in practice, and defend their consequences. The code you write focuses on the *business* logic: the *glue* connecting the selected components into a complete solution.

Two practices should guide the project::

- ✓ **State of the art**: Explore various options and choose the right one based on the requirements.
- ✓ **Proof of Concept**: Build a quick demo to verify that a component or algorithm works as expected.

⚠ Interconnecting two APIs is not what makes this project difficult. Assembling many parts into one product that runs on its own, survives its own failures, and can be handed to someone else is.

Your objectives

Build a software suite, inspired by IFTTT and Zapier, composed of:

- ✓ An application server to implement all the features of the project.
- ✓ A web client to use the application from your browser by querying the application server.
- ✓ A mobile client to use the application from your phone by querying the application server.



Any piece of software must answer real, practical needs of its end users: do not write code for the sake of writing code.

The expected work has two parts:

- ✓ a **common base** which every group delivers;
- ✓ a **selection of additional requirements**, chosen from a catalogue.



Additional requirements are deliberately expressed as client needs. Reformulating the need, choosing a solution and defending the trade-offs are part of your job.

Your Timeline

This project is divided into three milestones.

- ✓ **Plan** — investigate, experiment, decide and organize;
- ✓ **Minimum Viable Product** — deploy a coherent end-to-end product and confront the plan with reality;
- ✓ **Final Product** — deliver, prove and defend the finished product.

A milestone is evidence of progress, not a contract forbidding change. A decision may change when evidence invalidates it, provided the change, its reason and its consequences are documented.

Functional requirements

Users must be able to create and confirm an account, authenticate, subscribe to third-party **Services**, and create workflows that execute automatically.

Each **Service** offers at least two components:

- ✓ an **Action** is an event detected by the product;
- ✓ a **REAction** is a task performed by the product in response.

Authenticated user can compose:

- ✓ **AREA** interconnecting an Action to a REAction.
- ✓ workflows containing a single Action, several REActions and a condition.

Authentication and delegated access

The product supports:

- ✓ local authentication with a username and password;
- ✓ authentication through an external identity provider using **OpenID Connect (OIDC)**.

When a Service must act later on behalf of a user, the application uses the authorization mechanism provided by that Service, commonly **OAuth 2.0**, and stores the delegated credentials required for unattended hooks.



OIDC answers **who the user is**. OAuth 2.0 answers **what the application is authorized to do on the user's behalf**. A provider may participate in both flows, but they solve different problems.

The way external identities are linked to local users, and the lifecycle of delegated credentials, belong to your data and security design and must be documented.

Services, Actions and REActions

The application interconnects services with one another, so the authenticated user first selects the services they hold an account on and subscribes to them.

An Action is an event occurring inside the service that your product has to detect: a message posted, a file added, a deadline coming up. Detecting it is the job of a **hook**.

A REAction is a task your product performs inside the service when a hook has detected something.

The available Services, Actions and REActions are declared by the application server and exposed to the clients.

Examples include receiving a mail, detecting a new file, observing an issue, sending a message, storing a document or updating another service.



These examples are illustrative: nothing restricts you to the services used as examples in this subject.
A **service** you use yourself is a **service** you will test more often, and a demonstration built on something you care about is a better demonstration.



The value of a connector is the **real use case** it enables, not its presence in a count.

Workflows

The application allows to build and run workflow: a chain of nodes that the application server walks in order, once the Action has been detected.



Three kinds of node exist:

- ✓ A **trigger** node, representing the Action.
- ✓ A **reaction** node, representing REActions.

✓ An optional **condition** node sending the execution into one of two branches.



An AREA is the smallest workflow possible.

The common base deliberately bounds the problem without prescribing the implementation:

✓ the normal chain is linear;
✓ a condition has two successors;
✓ there is at most one condition and conditions are not nested;
✓ branches terminate the workflow and do not merge;
✓ workflows without a condition and workflows with a single reaction remain valid;
✓ the engine must demonstrate a meaningful workflow containing at least four reaction nodes, while the data model must not encode four as a structural maximum.

Structural validity is checked when the workflow is saved.

Decisions that are not specified here — for example whether an empty branch is valid — belong to the group and must be documented and applied consistently.



Each workflow you offer should save its user **real time or effort**. Chaining services together just because it is possible brings no value.

The part of the interface where users assemble their workflows is the **builder**.
Its form is free, but justified: a canvas, a list of steps, a form-based wizard, etc. are all acceptable.

The builder is **web only**: the nodes and the mapping of data between them are edited from the web client alone. The mobile client consumes and monitors workflows: it lists them, enables and disables them, and shows what they did. Building workflows from the mobile client is out of scope.

Data between nodes

Data travels from one node to the next. Every node publishes a declared set of values. Later nodes may reference earlier results through key paths such as `{{ trigger.subject }}` or `{{ node_2.file_id }}`.

We requires the following properties, not a particular implementation:

- ✓ each node type declares the keys it may publish;
- ✓ references outside that declaration are rejected when the workflow is saved;
- ✓ missing values at runtime are handled deterministically according to the group's documented model;
- ✓ substitution replaces data only and never evaluates user-supplied code.

Hooks

This essential element detects Actions and start the corresponding workflow without a client being open.

Hooks must avoid treating the same occurrence as a new event indefinitely. How events are identified, scheduled and persisted is an architectural decision to document and defend.



Technical requirements

Clients & Server

Mobile and web clients handle presentation and interaction. Business logic and rules belong to the application server which serves an *OpenAPI* description of its **HTTP API**, together with a browsable interface.

Clients and server are built in parallel by different members: this description is the contract between them. It covers every endpoint the clients call, error responses included. A test fails when the implementation and the description disagree.



Full-stack frameworks are strongly discouraged, because a single codebase blurs the boundary this project requires between clients and server, and makes splitting the work between members much harder.

The web client supports common browsers. The mobile client targets Android, and allows users to configure the application server's network address.



Targeting iOS as well is allowed, but should be considered as a bonus side quest.

Both clients meet the AA level of the *WCAG* standard. An **accessibility audit report** is committed, stating what was measured, with which tool, on which screens, what was fixed, and what was left unfixed and why.

Both clients are usable in at least two languages, on the whole main flow from registration to the creation of an **AREA**.

Reproducible construction

The complete product must be reproducibly buildable and runnable with **Docker Compose**. The exact internal decomposition of containers is a group decision; the subject does not require the architecture to mirror an historical three-container layout.

The delivered stack must make the server and web client reachable locally and must produce an installable Android application. The project documentation states the commands and resulting endpoints/artifacts.

Service discovery

While the stack is running, the application server exposes a machine-readable description of the Services, Actions and REActions available to clients. The endpoint must provide enough information for clients to discover their identifiers, descriptions and parameters without hard-coding the catalogue.

The exact schema is part of the group's modelling work. It must be documented, versionable and consistent with what the clients actually offer.

Additional features

The previous common base is not the whole project. Each group has to select additional features from a catalogue provided below.

This catalogue is built from a collection of quotes from our customers describing what they want to achieve, along with additional criteria that each feature must meet to be considered complete.



Customers verbatim may be imprecise. **Part of your job is to identify the real needs before any implementation**.

For the sake of clarity, each feature has been assigned:

- ✓ A unique identifier indicating its specialization domain
  software engineering, systems, security or artificial intelligence.
- ✓ A category based on its estimated difficulty, including testing and documentation, not just coding.
  1 = Small, 2 = Medium, 3 = Large.

Your selection is decided **and justified** at the first milestone, but can be revised once at the second milestone, and is then frozen and evaluated during the third milestone. Choose against your group's weaknesses rather than for comfort, and/or across several domains when possible: you will learn more and broaden your skills in the process.

If a feature turns out to be significantly more difficult than estimated, inform your local manager as soon as possible, while your selection can still be changed, and mention it again in your post-mortem, explaining what made it more complex and demanding than expected.

A feature is considered **delivered** only when all of its acceptance criteria can be observed. An ambitious feature that is not fully delivered may still produce useful engineering evidence for the project and individual defense, but it does not contribute its catalogue points.

Documents are evidence only when their claims can be defended or reproduced. The evaluator may select a claim and ask the group to demonstrate it live.

The catalogue

Each entry is an index.
Detailed information about each feature is provided in a separate document.

Id	Topic	Category
AI-EVAL	Model evaluation	2
AI-GUARD	Prompt injection defense	2
AI-MULTIMODAL	Multimodal inputs	2
AI-NODE	Autonomous AI node	3
AI-PROMPT	Prompt versioning	1
AI-REVIEW	Human validation	2
ENG-ADMIN	Administration area	2
ENG-ADR	Architecture decision records	1
ENG-ASYNC	Asynchronous execution	3
ENG-EXT	Extensible connectors	2
ENG-RETRY	Retries and idempotency	2
ENG-TEMPLATES	Shareable templates	3
SEC-AUTH	Authentication hardening	2
SEC-CRYPTO	Encryption at rest	2
SEC-DEPS	Dependency scanning	1
SEC-GDPR	Personal data compliance	2
SEC-SSRF	SSRF protection	2
SEC-THREAT	Threat modeling	3
SYS-BACKUP	Backup and restore	2
SYS-HEALTH	Health checks	1
SYS-K8S	Container orchestration	3
SYS-OBS	Observability	3
SYS-RELEASE	Release automation	2
SYS-SECRETS	Secrets management	1



Most of these features are independent from one another, and can be taken on their own.

Deliverables and expectations

We will take into account **not only the quality of your work but also its quantity**, measured by the number of functional features you implement in:

- ✓ the common base that every group has to deliver,
- ✓ the selection of additional features that each group chooses freely.

Let's consider the following quantities:

- ✓ M: The number of **members** in your group.
- ✓ S: The total number of server **services** available to both clients.
- ✓ A: The total number of **Actions** supported across all services.
- ✓ R: The total number of **REActions** supported across all services.
- ✓ D: The total estimated **difficulty** of additional features selected by your group.

Your delivery must satisfy the following conditions:

- ✓ $S \geq 1 + M$, at least half of them requiring the user to grant access through *OAuth2*
- ✓ $A + R \geq 2 * M + 2$
- ✓ $D \geq 2 * M$



Quality » quantity: Services, Actions and REActions picked only to reach the count, with no concrete use behind them, will not be valued.
Calculations are based on the less complete client: if one of the two clients (web or mobile) offers fewer features than the other.
Unfinished feature counts for nothing, but what it taught you is material for your post mortem and your defense.
Distribute features fairly: concentrating most of the effort on a few members puts both those people and the project at risk.

Documentation

The repository contains a `README.md` explaining setup, operation and the public API, plus a `HOWTOCONTRIBUTE.md` explaining how to extend Services, Actions, REActions and workflow nodes.

The technical documentation also covers:

- ✓ the workflow model and failure semantics;
- ✓ the persistence model and important data structures;
- ✓ the selected additional requirements and where each acceptance criterion can be observed;
- ✓ important architectural decisions, including alternatives considered, consequences and later revisions.



Each member must leave evidence supporting an individual architecture defense. At Plan, each member identifies at least one significant technical question to investigate or contribute to. By Final, each member must explain one architectural decision they materially contributed to, including the need, alternatives, evidence, decision, trade-offs, and observed result.
Repository history is supporting evidence; understanding and defending the decisions is what matters.



Milestone 1 - Plan

The objective is to turn uncertainty into investigated decisions, not to predict everything perfectly. Prove that your team knows what it will build, how, when and who. Experiment with several technology stacks and discuss different project management methods. Then, choose the ones that best fit your team.

One week after the start of the project, you must deliver:

- ✓ Three **technology selection documents** for the front-end, back-end, and persistence. Each report considers several technologies and evaluates them against relevant criteria (including security!), documents the comparison performed, and concludes with a concise, justified final choice, supported with **practical evidence**, such as exploratory code.
- ✓ The **selection of additional requirements**, with its learning rationale and ownership.
- ✓ The **significant architectural question** identified by each team member.
- ✓ A **task breakdown** of the whole project, where every task is actionable, scheduled, assigned and has its dependencies identified.
- ✓ The **project management method** and the various **tools** your team has chosen.

We expect that:

- ✓ The main lines of the project are understood.
- ✓ Every member has participated in exploration.
- ✓ Your selections (stack, methods, tools, features) are professionally justified.
- ✓ Your plan includes the common base, selected requirements, documentation, verification and the MVP deadline.

Milestone 2 - Minimum Viable Product

Build and deploy a **functional minimum viable product**. Keep the scope minimal, but make sure what you build is **usable end to end**. It does not have to be pretty.

Four weeks after the start of the project, you must deliver:

- ✓ A **reproducibly deployable product**.
- ✓ A **workflow running end to end**, containing a trigger, at least two reactions, and one condition.
- ✓ A first **technical documentation** of the core architecture.
- ✓ An **updated plan** and a written analysis of meaningful changes.
- ✓ **Observable progress** on at least one third of the selected catalogue points.
- ✓ Optionally, an updated **selection of additional requirements**: this is the last milestone at which it can change.

We expect that:

- ✓ Some tests cover important behaviors.
- ✓ On every contribution, automations run the linter, the tests and the build, and a failure is visible on that contribution.
- ✓ The *OpenAPI* description is served and covers the endpoints of your MVP.
- ✓ Every member explains their own contribution. The repository history supports that explanation, it does not replace it.
- ✓ Plan changes are expected: if anything changed, you explain the evidence that caused the change and the cost of the decision. An unchanged plan is not inherently better than a revised one.
- ✓ Meaningful progress has been made on several of your selected additional features: they are sufficiently advanced to be observed and demonstrated, rather than merely started.

Milestone 3 - Final Product

This last milestone closes the project: a finished product, the documentation that lets someone else take it over, and an honest look back at your own work.

Eventually, you must deliver:

- ✓ A **complete working product**, including **all the selected additional requirements**.
- ✓ Up-to-date **documentation**, including at least `README.md` and `HOWTOCONTRIBUTE.md`.
- ✓ An honest **accessibility audit report** for both clients, stating your method, tools, and findings.
- ✓ The **final planning artifacts** reflecting what actually happened.
- ✓ A **post-mortem** explaining what succeeded, what failed, what changed, what evidence changed your mind, and what you would do differently, including lessons from failed experiments.
- ✓ A **presentation** with a live **demonstration** and **technical defense**.

We expect that:



- ✓ Following your documentation allows anyone to build and run the project.
- ✓ You justify your choices regarding architecture, persistence, and data structures.
- ✓ Hooks run autonomously, and business logic is isolated behind abstractions.
- ✓ Both clients are responsive, visually consistent, and intuitive to navigate.
- ✓ Both clients meet *WCAG AA* standards and internationalization requirements.
- ✓ Every acceptance criterion of your additional features can be observed.
- ✓ Evidence for the additional security features follows the provided template.
- ✓ Some tests prove that the *OpenAPI* description matches the implementation.
- ✓ Unit and functional tests cover important success and failure paths.
- ✓ Automations run the linter, the tests and the build on every contribution, and deploy the product without manual steps.
- ✓ The presentation clearly communicates the product and its context.
- ✓ You demonstrate and explain a meaningful workflow with at least four reaction nodes.
- ✓ Visual polish supports communication but does not replace technical evidence.
- ✓ Your defense focuses on architectural decisions, trade-offs, evidence, limits, and individual understanding.



Evaluators must be able to verify any of your claims live, and can ask you to demonstrate any feature of your product.



Keep a fallback recording for demonstrations relying on unreliable third parties.


EPITECH